Numerical solution of the SIR epidemic model using Euler and Runge-Kutta Methods

Riley Marriott, James, James, Andre

March 2026

Contents

1 Abstract **4**

2 Introduction **4**

3 Theory **4**
3.1 Ordinary Differential Equations . . . 4
3.2 Derivation of Numerical Approximation Framework . . . 5
3.3 Euler's Method . . . 5
3.3.1 Error Analysis of Euler's Method . . . 5
3.4 Runge-Kutta Methods . . . 5
3.4.1 Second-Order Runge–Kutta Method (RK2) . . . 5
3.4.2 Error Analysis of RK2 . . . 6
3.4.3 Fourth-Order Runge–Kutta Method (RK4) . . . 6
3.4.4 Error Analysis of RK4 . . . 6
3.5 The SIR Model . . . 6
3.6 Basic Reproduction Number . . . 7
3.7 Conservation of Population . . . 7
3.8 Implications for Numerical Methods . . . 7

4 Numerical Methods **7**
4.1 Euler Method Implementation . . . 7
4.2 Second-Order Runge–Kutta Method (RK2) . . . 8
4.2.1 Error Analysis of RK2 . . . 8
4.3 Fourth-Order Runge–Kutta Method (RK4) . . . 9
4.3.1 Error Analysis of RK4 . . . 10
4.4 Algorithm General and Sample Flowcharts . . . 10
4.5 Implementation Summary . . . 11

5 Examples and Applications **11**
5.1 Benchmark Problem . . . 11
5.2 Computational Time Comparison . . . 13
5.3 SIR Model Simulation . . . 14
5.4 Validation of the Numerical Solution . . . 15
5.5 Overall Comparison . . . 15

6 Conclusion **15**

7 Extensions **15**

8 Bibliography **16**

9 Appendix **16**

10 Notes **16**

11 Marking **16**

1 Abstract

Ordinary Differential Equations (ODEs) are used to model dynamic processes in many disciplines, including engineering, physics, and biology. In epidemiology, systems of ODEs are commonly employed to simulate the spread of infectious diseases within a population. However, many realistic models are nonlinear and do not admit simple analytical solutions, making numerical methods essential for approximating their behaviour. Among the most widely used approaches are the Runge–Kutta (RK) methods, which offer improved accuracy compared to simpler methods such as Euler's method.

This paper investigates the performance of several numerical methods for solving ordinary differential equations, with a particular focus on the Euler method, the second-order Runge–Kutta method (RK2), and the fourth-order Runge–Kutta method (RK4). The methods are first applied to a simple ODE with a known analytical solution to evaluate convergence behaviour and global error as a function of step size. They are then applied to the nonlinear Susceptible–Infected–Recovered (SIR) epidemic model to simulate the spread of disease within a population. The numerical solutions are evaluated in terms of accuracy, stability, and computational efficiency, providing insight into the relative performance of each method when modelling complex real-world systems. The results highlight the trade-off between numerical accuracy and computational cost, illustrating the advantages of higher-order Runge–Kutta methods for modelling nonlinear dynamical systems.

2 Introduction

Ordinary Differential Equations (ODEs) describe systems in which variables evolve over time according to specified rates of change. These equations are fundamental in many disciplines, including engineering (fluid flow, heat transfer systems), physics (oscillations and motion), and biology (population dynamics and epidemiology).

In many practical situations, ODE systems are nonlinear, coupled, and do not admit simple analytical solutions. As a result, numerical methods must be implemented to approximate their behaviour. These methods discretise the time domain and compute approximate solutions at discrete points, allowing complex systems to be analysed and predicted.

Within epidemiology, modelling the spread of infectious disease is a dynamic problem of significant importance. Mathematical models help predict outbreaks, estimate peak infection levels, and inform public health decisions. One of the most widely used models is the SIR model, which divides a population into susceptible, infected, and recovered compartments.

This report investigates the performance of several numerical methods for solving ODEs, with particular focus on Euler's method, the second-order Runge–Kutta method (RK2), and the fourth-order Runge–Kutta method (RK4). The methods are first evaluated using a simple ODE with a known analytical solution, before being applied to the nonlinear SIR model. The accuracy, stability, and computational efficiency of each method are compared to determine their suitability for different types of problems.

3 Theory

3.1 Ordinary Differential Equations

An Ordinary Differential Equation (ODE) describes the relationship between a function and its derivatives with respect to a single independent variable (in this case, time). A general first-order Initial Value Problem (IVP) is given by:

$$\frac{dy}{dt} = f(t, y), \quad y(t_0) = y_0, \tag{1}$$

where $f(t, y)$ is a known function and $y(t)$ represents the unknown solution.

In many practical applications, particularly when $f(t, y)$ is nonlinear, analytical solutions can be extremely difficult or impossible to obtain. As a result, the solution must be approximated numerically by discretising time into steps of size h, producing a sequence of approximations $\{y_n\}$ at discrete points $t_n = t_0 + nh$.

3.2 Derivation of Numerical Approximation Framework

To construct a numerical scheme, we begin by integrating both sides of Equation (1) over a small interval $[t_n, t_{n+1}]$:

$$y(t_{n+1}) = y(t_n) + \int_{t_n}^{t_{n+1}} f(t, y(t))\, dt. \tag{2}$$

This integral cannot generally be evaluated analytically because the exact solution $y(t)$ is unknown. Therefore, the problem reduces to approximating this integral. The different numerical methods considered in this report—Euler's method, RK2, and RK4—can all be interpreted as different approaches to approximating this integral, with varying levels of accuracy and computational cost.

3.3 Euler's Method

The simplest approach to approximating the integral is Euler's method. We approximate $f(t, y)$ as constant over the interval using its value at the beginning of the step:

$$\int_{t_n}^{t_{n+1}} f(t, y(t))\, dt \approx h f(t_n, y_n). \tag{3}$$

Substituting into Equation (2) gives Euler's method:

$$y_{n+1} = y_n + h f(t_n, y_n). \tag{4}$$

The solution is approximated by assuming that the slope remains constant over each time step, using the derivative evaluated at the beginning of the interval. Geometrically, this corresponds to advancing the solution along the tangent line at (t_n, y_n). While this approach is simple and computationally inexpensive, it neglects curvature in the solution, leading to rapid accumulation of error. Consequently, Euler's method is only first-order accurate and requires sufficiently small step sizes to maintain stability and accuracy.

3.3.1 Error Analysis of Euler's Method

Using a Taylor series expansion of the true solution,

$$y(t_{n+1}) = y(t_n) + h y'(t_n) + \frac{h^2}{2} y''(\xi), \tag{5}$$

for some $\xi \in [t_n, t_{n+1}]$, Euler's method neglects higher-order terms. The local truncation error is therefore $\mathcal{O}(h^2)$, which leads to a global error of $\mathcal{O}(h)$.

3.4 Runge-Kutta Methods

Runge–Kutta methods improve accuracy by evaluating $f(t, y)$ at multiple points within each time step, providing a more accurate approximation of the integral in Equation (2).

3.4.1 Second-Order Runge–Kutta Method (RK2)

A second-order approximation can be obtained by averaging the slope over the interval:

$$k_1 = f(t_n, y_n), \tag{6}$$

$$k_2 = f(t_n + h, y_n + h k_1), \tag{7}$$

$$y_{n+1} = y_n + \frac{h}{2}(k_1 + k_2). \tag{8}$$

The RK2 method improves upon Euler's method by incorporating information about the slope at both the beginning and end of the interval. This allows the method to better capture variation in the slope across the step, improving the approximation of the solution. As a result, RK2 achieves second-order accuracy and provides significantly improved results compared to Euler's method, with only a moderate increase in computational cost.

3.4.2 Error Analysis of RK2

The RK2 method has a local truncation error of $\mathcal{O}(h^3)$ and a global error of $\mathcal{O}(h^2)$, allowing larger step sizes to be used while maintaining good accuracy.

3.4.3 Fourth-Order Runge–Kutta Method (RK4)

A further refinement is given by the fourth-order Runge–Kutta method:

$$k_1 = f(t_n, y_n), \tag{9}$$

$$k_2 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_1\right), \tag{10}$$

$$k_3 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_2\right), \tag{11}$$

$$k_4 = f(t_n + h, y_n + hk_3), \tag{12}$$

$$y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4). \tag{13}$$

The RK4 method evaluates the slope at four points within each time step and combines them using a weighted average. This enables the method to accurately capture higher-order behaviour of the solution, including curvature and variation in the slope. As a result, RK4 achieves fourth-order accuracy and typically produces highly accurate results even for relatively large step sizes.

3.4.4 Error Analysis of RK4

RK4 has a local truncation error of $\mathcal{O}(h^5)$ and a global error of $\mathcal{O}(h^4)$. Although it requires more function evaluations per step, its superior accuracy often leads to greater overall efficiency compared to lower-order methods.

3.5 The SIR Model

The Susceptible–Infected–Recovered (SIR) model describes the spread of infectious disease within a closed population. The total population, N, is divided into three groups:

- $S(t)$: susceptible individuals,
- $I(t)$: infected individuals,
- $R(t)$: recovered individuals.

The governing equations are

$$\frac{dS}{dt} = -\beta SI, \tag{14}$$

$$\frac{dI}{dt} = \beta SI - \gamma I, \tag{15}$$

$$\frac{dR}{dt} = \gamma I, \tag{16}$$

where β is the transmission rate and γ is the recovery rate.

The term βSI arises from the assumption that the rate of infection is proportional to both the number of susceptible and infected individuals, reflecting random interactions within the population. The recovery term γI assumes that infected individuals recover at a constant rate.

3.6 Basic Reproduction Number

The basic reproduction number is defined as

$$R_0 = \frac{\beta}{\gamma}, \tag{17}$$

and represents the expected number of secondary infections produced by a single infected individual in a fully susceptible population. If $R_0 > 1$, the infection spreads, whereas if $R_0 < 1$, the disease dies out.

3.7 Conservation of Population

Summing the governing equations yields

$$\frac{d}{dt}(S + I + R) = 0, \tag{18}$$

which implies

$$S(t) + I(t) + R(t) = N. \tag{19}$$

This conservation law provides an important validation check for numerical solutions.

3.8 Implications for Numerical Methods

The SIR system is nonlinear and coupled, making it sensitive to numerical errors. Poor numerical schemes or excessively large step sizes can produce non-physical results, such as negative population values or violation of the conservation law. Therefore, it is essential to use numerical methods that are both accurate and stable when simulating epidemic dynamics.

4 Numerical Methods

In this section, the numerical schemes derived in the previous section are translated into practical algorithms. While the theory focused on the approximation of the integral form of the governing equations, the purpose here is to describe how these methods are implemented and applied in a computational setting.

Each method advances the solution iteratively using a fixed time step h, allowing the solution to be approximated over a specified interval. The differences between the methods arise from how the integral in Equation (2) is approximated, which directly impacts both accuracy and computational cost.

4.1 Euler Method Implementation

Euler's method provides the simplest numerical implementation of the integral approximation introduced in the theory section. The solution is advanced using the slope evaluated at the beginning of each time step:

$$y_{n+1} = y_n + hf(t_n, y_n).$$

The method is implemented iteratively, starting from the initial condition y_0. At each time step, the derivative function is evaluated once, and the solution is updated accordingly.

Due to its simplicity, Euler's method is computationally efficient and easy to implement. However, because it assumes the slope remains constant over the interval, it introduces relatively large numerical errors unless a sufficiently small step size is used. As a result, it is primarily useful as a baseline method for comparison with higher-order schemes.

Figure 1 shows the general algorithmic structure of Euler's method. The method requires only one slope evaluation per step, which explains its low computational cost, but this simplicity comes at the expense of lower accuracy.

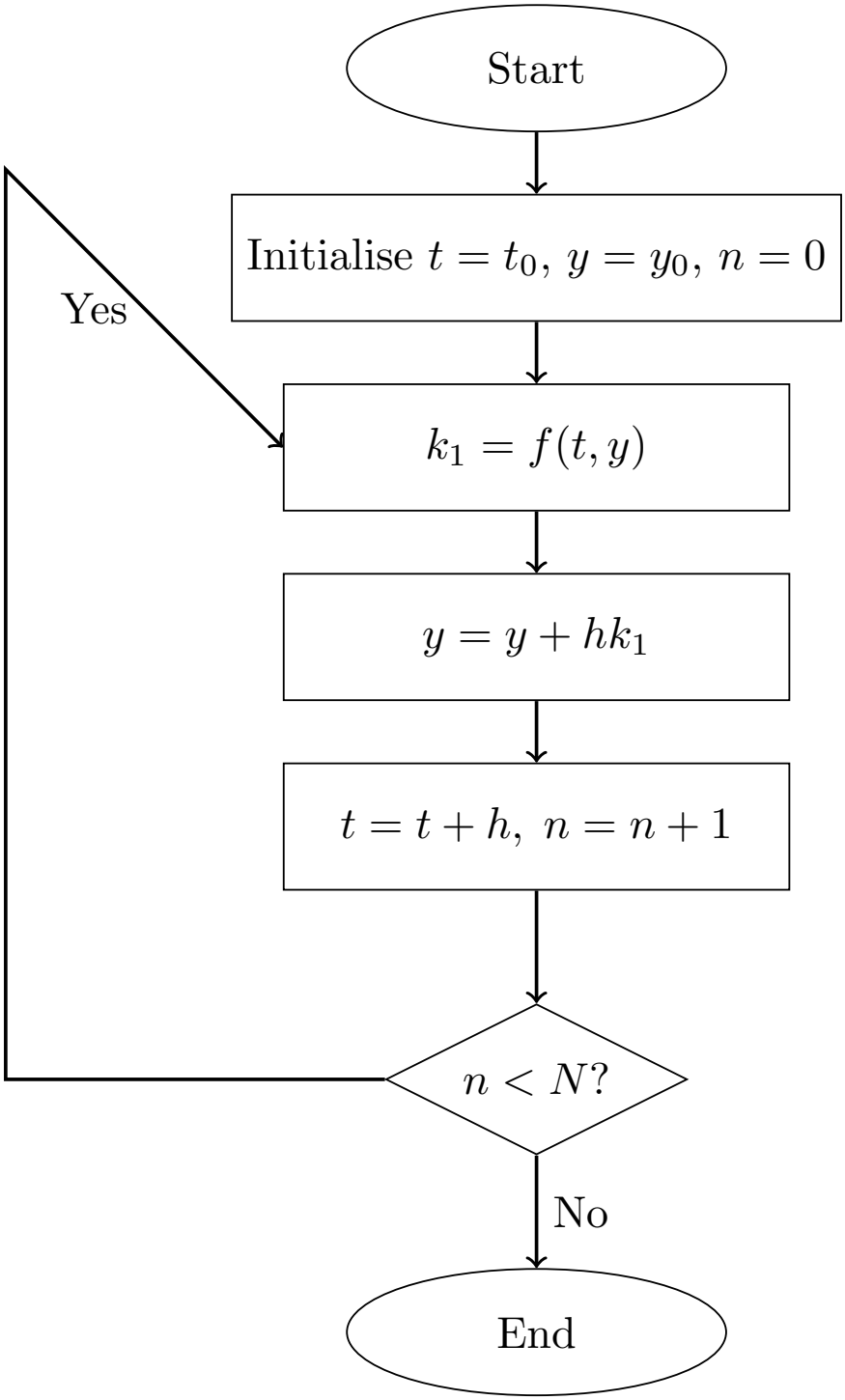


Figure 1: General flowchart for the Euler algorithm.

4.2 Second-Order Runge–Kutta Method (RK2)

The second-order Runge–Kutta (RK2) method improves upon Euler's method by incorporating information about the slope at multiple points within each time step, rather than relying solely on the slope at the beginning of the interval. This results in a more accurate approximation of the integral in Equation (2), and consequently a better approximation of the solution.

To construct the method, we again consider the integral form:

$$y(t_{n+1}) = y(t_n) + \int_{t_n}^{t_{n+1}} f(t, y(t)), dt. \tag{20}$$

Instead of approximating the integrand as constant (as in Euler's method), RK2 approximates the average slope over the interval by evaluating the derivative at both the beginning and the end of the step. First, an initial estimate of the slope is computed:

$$k_1 = f(t_n, y_n). \tag{21}$$

This is used to predict a provisional value at the end of the interval:

$$y_n^* = y_n + hk_1. \tag{22}$$

A second slope is then evaluated at this predicted point:

$$k_2 = f(t_n + h, y_n^*). \tag{23}$$

The solution is then advanced using the average of these two slopes:

$$y_{n+1} = y_n + \frac{h}{2}(k_1 + k_2). \tag{24}$$

Geometrically, this method can be interpreted as averaging the slope at the beginning and the end of the interval, thereby accounting for variation in the derivative across the step.

Figure 2 shows the general implementation of the RK2 method. Compared with Euler's method, the additional prediction and second slope evaluation allow the method to better approximate the average slope over the interval.

4.2.1 Error Analysis of RK2

To analyse the error, we again consider the Taylor series expansion of the exact solution about t_n:

$$y(t_{n+1}) = y(t_n) + hy'(t_n) + \frac{h^2}{2}y''(t_n) + \frac{h^3}{6}y'''(\xi), \tag{25}$$

for some $\xi \in [t_n, t_{n+1}]$.

We now expand the RK2 update. Using $k_1 = f(t_n, y_n) = y'(t_n)$, and noting that

$$k_2 = f(t_n + h, y_n + hk_1), \tag{26}$$

we expand k_2 about (t_n, y_n):

$$k_2 = y'(t_n) + h\frac{\partial f}{\partial t} + hy'(t_n)\frac{\partial f}{\partial y} + \mathcal{O}(h^2). \tag{27}$$

Combining terms, the RK2 update becomes

$$y_{n+1} = y_n + hy'(t_n) + \frac{h^2}{2}y''(t_n) + \mathcal{O}(h^3). \tag{28}$$

Comparing with the exact Taylor expansion, we see that RK2 correctly reproduces all terms up to order h^2, and neglects terms of order h^3 and higher. Therefore, the local truncation error is

$$\tau_n = \mathcal{O}(h^3). \tag{29}$$

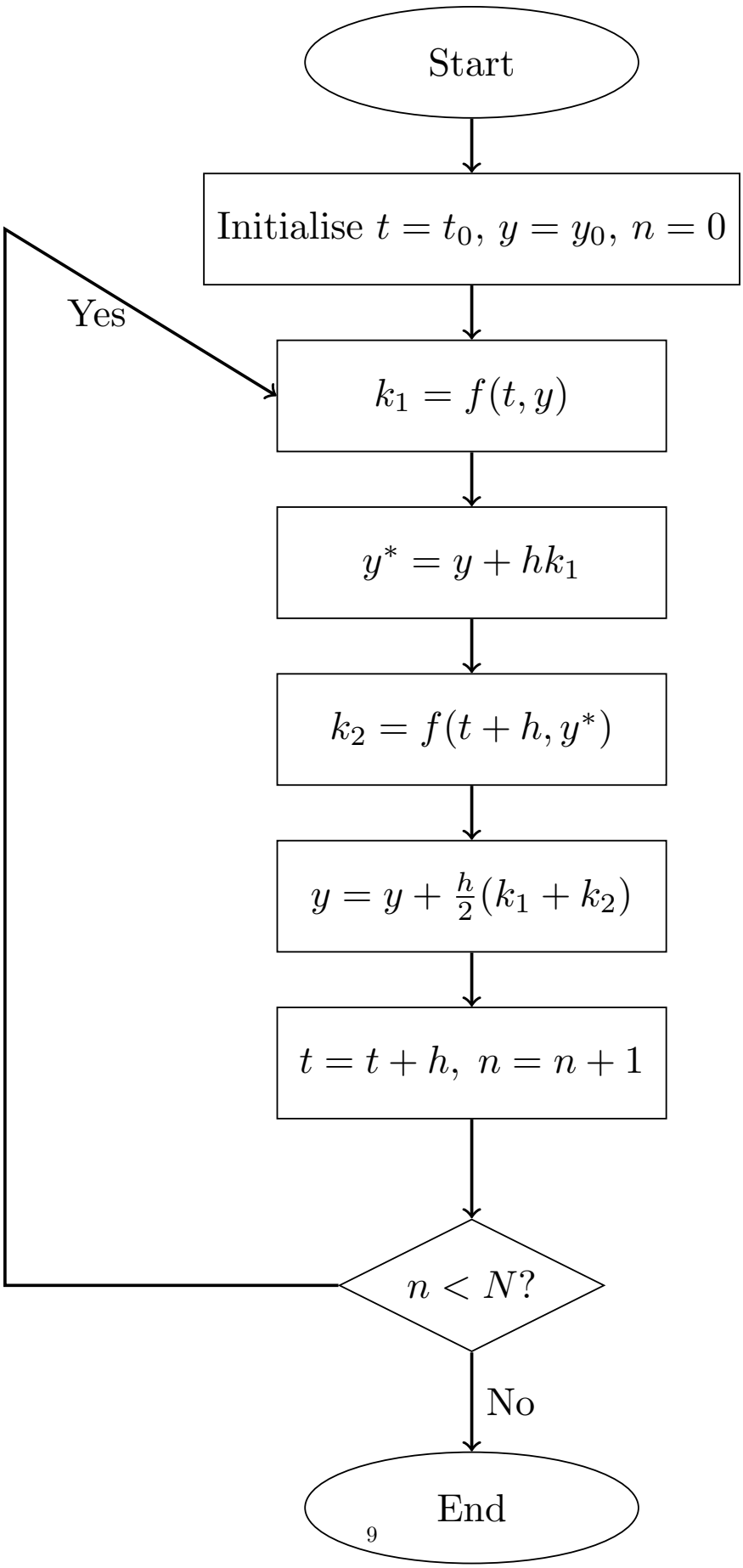


Figure 2: General flowchart for the RK2 algorithm.

Over $N \sim \frac{1}{h}$ steps, this accumulates to a global error of

$$E_N = \mathcal{O}(h^2). \tag{30}$$

The RK2 method is second-order accurate. This represents a significant improvement over Euler's method, as the error decreases quadratically with step size rather than linearly. In practical terms, this allows for larger step sizes to be used while maintaining a comparable level of accuracy. This behaviour will be examined later in the benchmark problem through convergence results.

4.3 Fourth-Order Runge–Kutta Method (RK4)

The fourth-order Runge–Kutta (RK4) method further refines the approximation of the integral in Equation (2) by evaluating the slope at multiple intermediate points within each time step and combining them in a weighted average. This approach provides a highly accurate estimate of the average slope over the interval, allowing the method to capture both curvature and higher-order behaviour of the solution.

Starting again from the integral formulation,

$$y(t_{n+1}) = y(t_n) + \int_{t_n}^{t_{n+1}} f(t, y(t)), dt, \tag{31}$$

the RK4 method approximates the integral using four slope evaluations.

First, the slope at the beginning of the interval is computed:

$$k_1 = f(t_n, y_n). \tag{32}$$

A second slope is evaluated at the midpoint using k_1:

$$k_2 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_1\right). \tag{33}$$

A third slope is again evaluated at the midpoint, now using k_2:

$$k_3 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_2\right). \tag{34}$$

Finally, a fourth slope is evaluated at the end of the interval using k_3:

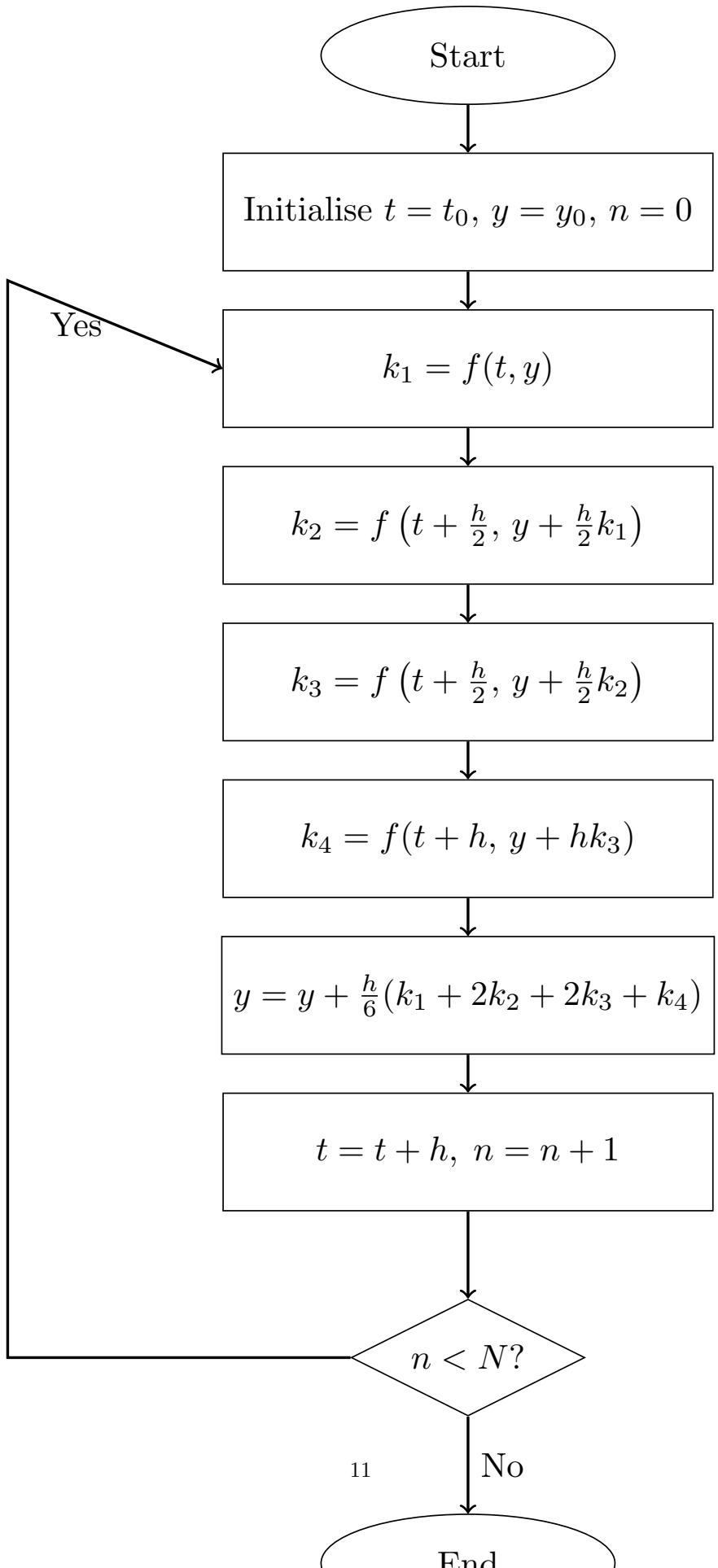
$$k_4 = f(t_n + h, y_n + hk_3). \tag{35}$$

These slopes are combined in a weighted average to advance the solution:

$$y_{n+1} = y_n + \frac{h}{6}\left(k_1 + 2k_2 + 2k_3 + k_4\right). \tag{36}$$

The weighting reflects the relative importance of each slope in approximating the integral, with greater weight placed on the midpoint evaluations. Geometrically, RK4 can be interpreted as constructing a highly accurate estimate of the average slope over the interval by sampling the curve at chosen points. This allows the method to closely track the true trajectory of the solution even if the derivative varies significantly within a single step.

Figure 3 illustrates the structure of the RK4 method. The four slope evaluations and weighted average provide a much more accurate estimate of the integral over each time step, which explains the improved convergence behaviour observed later in the benchmark results.


Start
Initialise $t = t_0,\ y = y_0,\ n = 0$
Yes
$k_1 = f(t, y)$
$k_2 = f\left(t + \frac{h}{2},\ y + \frac{h}{2}k_1\right)$
$k_3 = f\left(t + \frac{h}{2},\ y + \frac{h}{2}k_2\right)$
$k_4 = f(t + h,\ y + hk_3)$
$y = y + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$
$t = t + h,\ n = n + 1$
$n < N?$
No
End

4.3.1 Error Analysis of RK4

To analyse the error, we again consider the Taylor series expansion of the exact solution:

$$y(t_{n+1}) = y(t_n) + hy'(t_n) + \frac{h^2}{2}y''(t_n) + \frac{h^3}{6}y'''(t_n) + \frac{h^4}{24}y^{(4)}(t_n) + \frac{h^5}{120}y^{(5)}(\xi), \tag{37}$$

for some $\xi \in [t_n, t_{n+1}]$.

The RK4 method constructs its update by combining the four slope evaluations. Expanding each k_i about (t_n, y_n) and substituting into the update formula yields

$$y_{n+1} = y_n + hy'(t_n) + \frac{h^2}{2}y''(t_n) + \frac{h^3}{6}y'''(t_n) + \frac{h^4}{24}y^{(4)}(t_n) + \mathcal{O}(h^5). \tag{38}$$

The RK4 update matches the Taylor expansion of the true solution up to terms of order h^4, with discrepancies only appearing at order h^5. The local truncation error is therefore

$$\tau_n = \mathcal{O}(h^5). \tag{39}$$

Over $N \sim \frac{1}{h}$ steps, this leads to a global error of

$$E_N = \mathcal{O}(h^4). \tag{40}$$

Therefore, the RK4 method is fourth-order accurate. This implies that the error decreases very rapidly as the step size is refined. In practical terms, halving the step size reduces the global error by approximately a factor of 16, which is a substantial improvement over lower-order methods. Consequently, RK4 is able to achieve high accuracy with relatively large step sizes, making it particularly well-suited to nonlinear systems such as the SIR model. This improved accuracy makes higher-order methods particularly suitable for nonlinear and coupled systems, where small numerical errors can significantly affect the solution behaviour. This improved performance will also be reflected later in both the benchmark convergence plots and the computational time comparison.

4.4 Algorithm General and Sample Flowcharts

To further demonstrate how the algorithms operate in practice, worked flowcharts were constructed for RK2 and RK4 using the differential equation

$$\frac{dy}{dx} = e^{2x}, \qquad y(0) = 1,$$

with step size $h = 0.1$. These examples complement the general flowcharts presented earlier and show how the numerical updates are carried out step-by-step, providing a direct illustration of the difference in accuracy between the two methods.

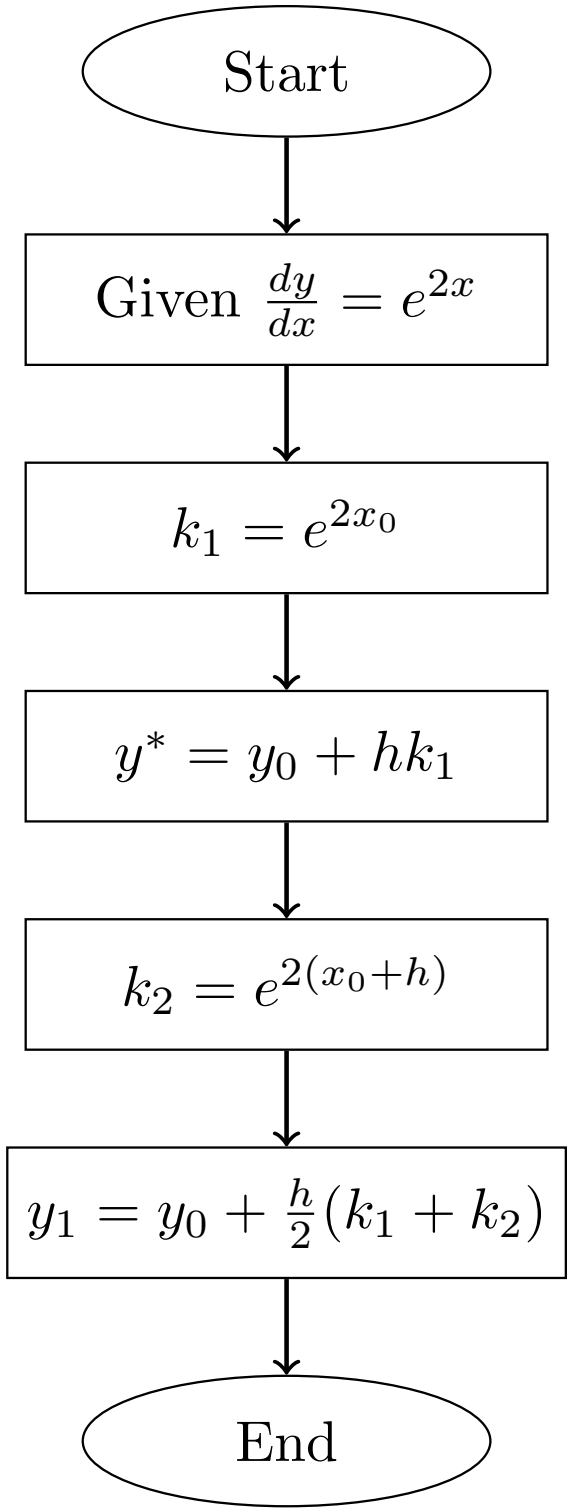
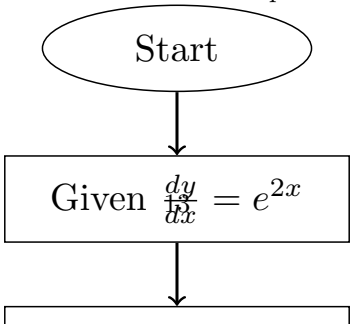


Figure 4: *
RK2 Worked Example



The worked examples demonstrate that RK4 produces a much smaller numerical error than RK2 for the same step size. This is consistent with the theoretical error analysis, where RK2 exhibits $\mathcal{O}(h^2)$ global error and RK4 exhibits $\mathcal{O}(h^4)$ global error. Together with the general flowcharts presented earlier, these examples provide a direct link between the mathematical derivation of the methods and their practical implementation in code.

4.5 Implementation Summary

Although the methods differ in their mathematical formulation, they are all implemented using a time-stepping procedure. For a given step size h, the total number of steps is

$$N = \frac{T - t_0}{h}.$$

At each step, the solution is updated using the formulas derived above. The primary difference between the methods lies in the number of function evaluations required:

- Euler method: 1 evaluation per step,
- RK2 method: 2 evaluations per step,
- RK4 method: 4 evaluations per step.

This difference in computational effort per step is important when comparing runtime, and will be reflected later in the computational time graph.

For the SIR model, the methods are extended to a system of equations by representing the solution as a vector $\mathbf{y} = [S, I, R]^T$. The update formulas are then applied component-wise, allowing all variables to be advanced simultaneously.

This implementation framework allows direct comparison of accuracy, convergence, and computational cost across the different numerical methods.

5 Examples and Applications

To evaluate the performance of the numerical methods, two types of problems were considered: a benchmark differential equation with a known analytical solution and the nonlinear SIR epidemic model. The benchmark problem allows the numerical error and convergence behaviour to be quantified, while the SIR model demonstrates the application of the methods to a realistic coupled nonlinear system.

5.1 Benchmark Problem

The benchmark problem considered is

$$\frac{dy}{dt} = -2y, \quad y(0) = 1,$$

which has the exact solution

$$y(t) = e^{-2t}.$$

This problem is particularly useful because the exact solution is known, allowing direct comparison with the numerical approximations. Euler's method, RK2, and RK4 were applied over the interval $[0, 2]$ using a range of step sizes.

To assess accuracy, the maximum global error was computed as

$$E_h = \max_{t_n} \left| y_{\text{exact}}(t_n) - y_n \right|.$$

The observed order of convergence was estimated using

$$p \approx \frac{\log(E_h/E_{h/2})}{\log 2}.$$

Figure 6 compares the numerical and exact solutions for a representative step size. Euler's method shows the largest deviation from the exact solution due to its first-order accuracy. RK2 provides a noticeable improvement by incorporating additional slope information, while RK4 produces results that are almost indistinguishable from the exact solution.

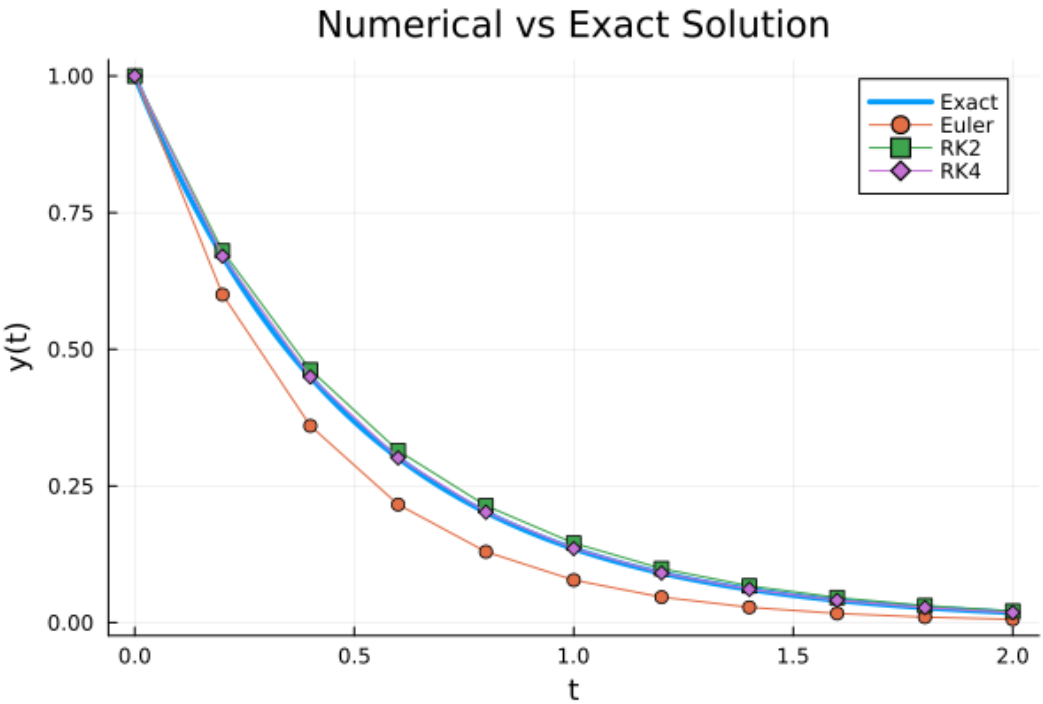


Figure 7: Comparison of Euler, RK2, and RK4 solutions with the exact solution for the benchmark problem $y' = -2y$.

Figure 7 shows the convergence behaviour of the three methods on a log–log scale. The slopes of the curves clearly match the theoretical error orders derived earlier, with Euler exhibiting first-order convergence, RK2 exhibiting second-order convergence, and RK4 exhibiting fourth-order convergence.

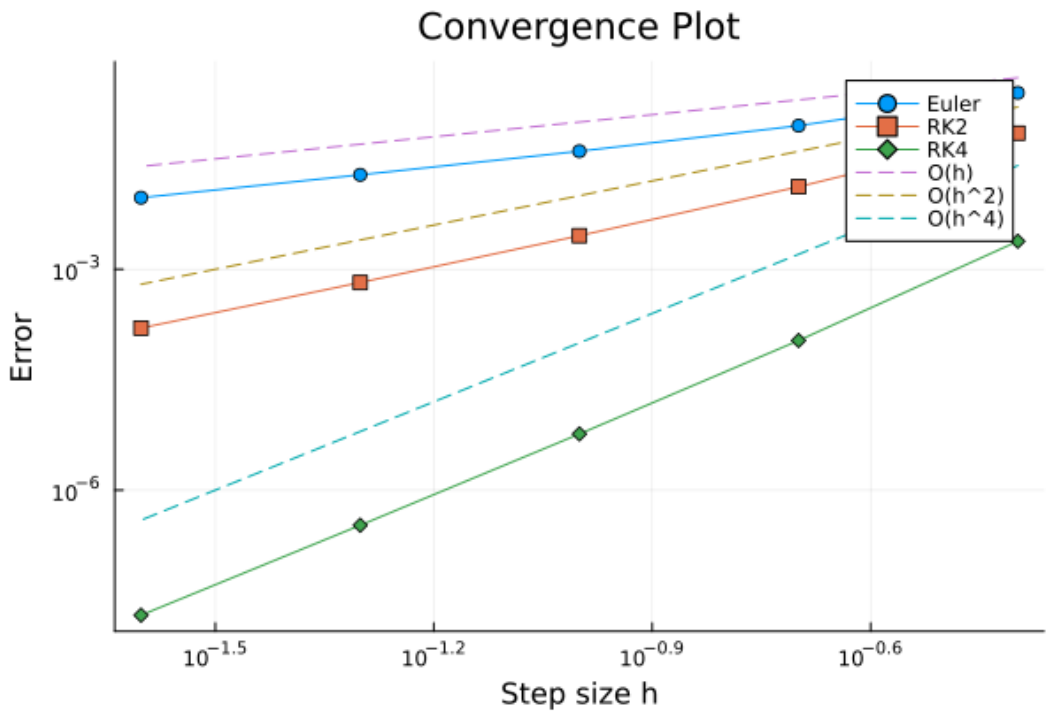


Figure 8: Log–log convergence plot showing the dependence of maximum global error on step size for Euler, RK2, and RK4.

These results confirm that higher-order methods provide significantly improved accuracy for a given step size. While Euler's method is simple and computationally efficient, it requires very small step sizes to achieve acceptable accuracy. In contrast, RK4 achieves high accuracy even for relatively large step sizes.

5.2 Computational Time Comparison

In addition to accuracy, the computational cost of each numerical method was assessed using the benchmark problem. The average runtime of Euler's method, RK2, and RK4 was measured over repeated simulations using the same step size.

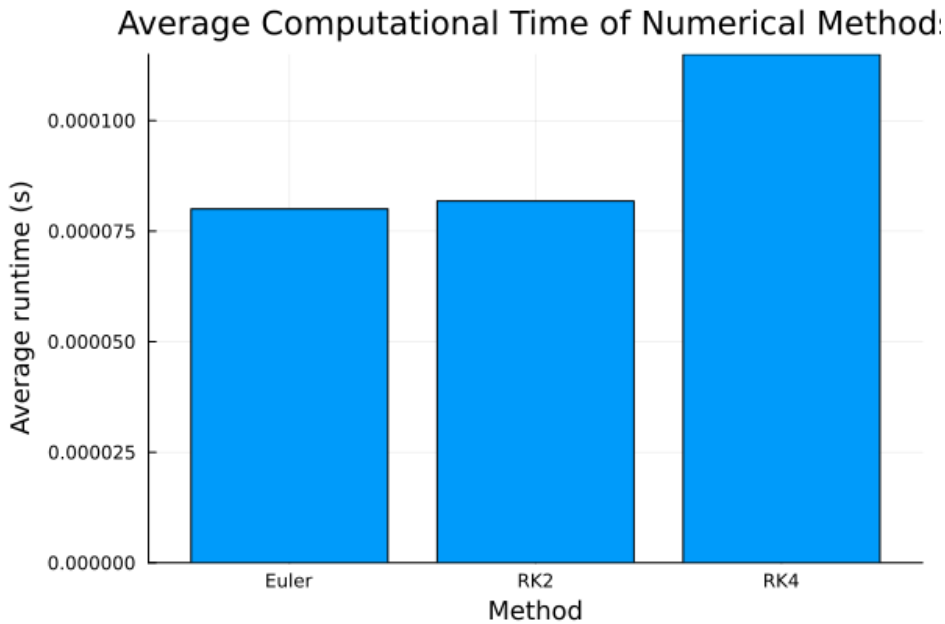


Figure 9: Average computational time for Euler, RK2, and RK4 on the benchmark problem.

The timing results are consistent with the structure of the methods. Euler's method is the fastest because it evaluates the derivative only once per step, while RK2 and RK4 require two and four evaluations respectively. Although RK4 has the highest computational cost for a fixed step size, its superior convergence behaviour means that it can achieve a much smaller error with fewer steps overall. Therefore, the most computationally expensive method per step is not necessarily the least efficient method in practice.

5.3 SIR Model Simulation

The SIR model was solved numerically using the RK4 method with parameters

$$\beta = 0.35, \qquad \gamma = 0.10,$$

and initial conditions

$$S_0 = 0.95, \qquad I_0 = 0.05, \qquad R_0 = 0.$$

The simulation was performed over the interval $[0, 160]$ with step size $h = 0.5$. The resulting compartment histories are shown in Figure 9.

Initially, the infected population increases rapidly as the disease spreads through the susceptible population. As the number of susceptible individuals decreases and recovery becomes dominant, the infected population reaches a peak before declining. Meanwhile, the recovered population increases monotonically and the susceptible population decreases continuously.

The numerical results indicate that the infected population reaches a maximum of approximately $I_{\max} \approx 0.36$ at around $t \approx 25$. This behaviour is consistent with the theoretical expectation, since the basic reproduction number is

$$R_0 = \frac{\beta}{\gamma} = 3.5 > 1,$$

indicating that the infection spreads through the population before eventually declining.

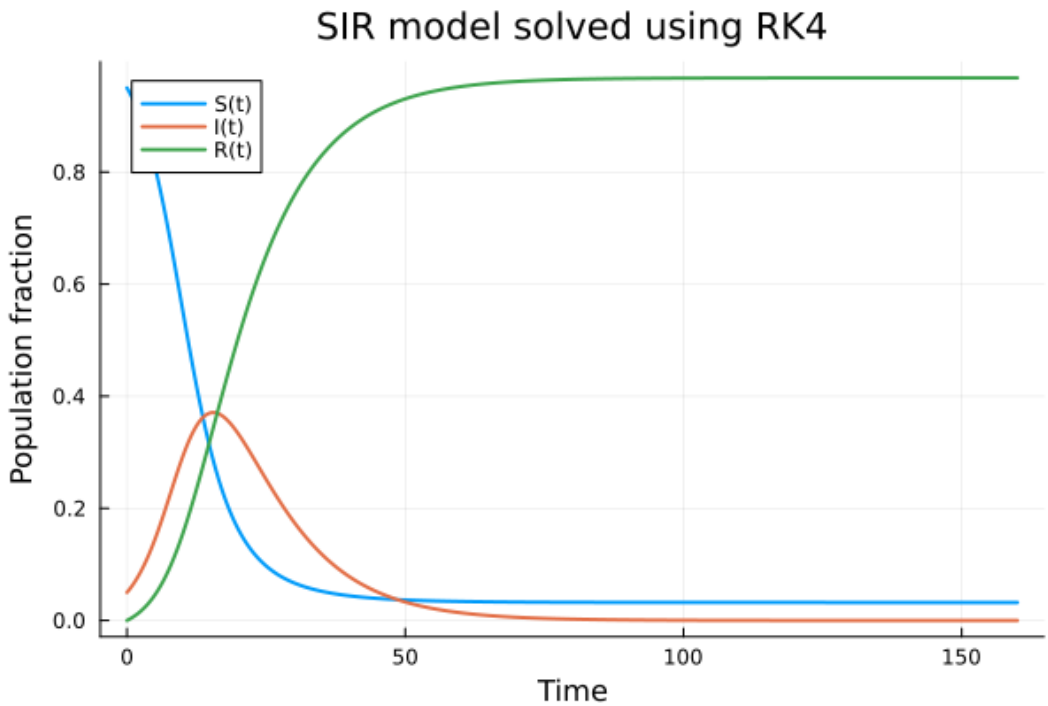


Figure 10: Time evolution of the susceptible, infected, and recovered populations obtained using the RK4 method.

5.4 Validation of the Numerical Solution

An important property of the SIR model is conservation of total population:

$$S(t) + I(t) + R(t) = 1.$$

The numerical solution satisfied this condition with a maximum deviation of less than 10^{-12}, indicating excellent numerical stability and negligible accumulated error. This provides strong validation of both the implementation and the suitability of RK4 for this nonlinear system.

5.5 Overall Comparison

The benchmark and SIR results together demonstrate the trade-off between computational cost and accuracy. Euler's method is simple and fast but relatively inaccurate. RK2 improves the approximation significantly with a moderate increase in computational cost. RK4 is the most computationally expensive per step, but its superior convergence behaviour and stability make it the most effective method overall.

For nonlinear systems such as the SIR model, where errors can propagate through coupled variables, the improved robustness of higher-order methods becomes particularly important.

6 Conclusion

Short summary of findings and enumeration of prospective work

7 Extensions

Final statement about relevance and impact

8 Bibliography

List of references and sources of information that must be cited in main text

9 Appendix

10 Notes

• the main body consists of sections 2-6 (intro to conclusion) and must be between 1500-2500 words (that is the equivalent of a 3-5 pages essay). The Appendix can be as long as desired but it will not be graded.
• the submitted file format must be a single PDF document. Jupyter Notebooks with extension .ipynb may be included as appendices (see below). Any other formats will be rejected and not be graded.
• the article must contain at least two (2) figures with caption (description of what the figure conveys) and properly referenced in the main text.
• any source code written to perform the numerical computations and to produce the article's results.
• the file format must either be .jl text files or Jupyter Notebook .ipynb.
• no Julia packages other than Plots is allowed.
• source code must be adequately documented.
• source code must run without error.
• it must be possible to run all computations on a standard laptop in a reasonable amount of time.
• an authors declaration: a brief description of each author's roles and contributions: the document must be signed by all members of the group.

11 Marking

50 on the report's content:
- 20 on the mathematical substance, definitions, the correctness of derivations, technical explanations
- 10 on the choice of examples/applications, appropriate use of figures, level of sophistication
- 10 on the clarity, structure and story-line, motivation and context, conclusion and impact
- 10 on novelty and experimentation

20 on the report's form:
- 10 for tidiness, neatness, appropriate use of fonts, typesetting of mathematics, symbols, etc.
- 10 for visuals, figures and overall aesthetics.

30 about the source code:
- 10 for code that runs correctly and produces desired results
- 10 for level of sophistication, speed/efficiency considerations
- 10 for readability, good coding style/practice and documentation